МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №7

з дисципліни «Функціональне програмування»

на тему: «functools та operator»

Варіант 6: «Замовлення інтернет-магазину»

Виконав:
студент групи ФЕП-23
Корж Олексій

Перевірив:
проф. Свелеба С. А.

Львів – 2026

1. Мета роботи

Опанувати засоби стандартних бібліотек functools та operator: часткове застосування функцій (partial), поліморфну диспетчеризацію (@singledispatch), автоматичну генерацію методів порівняння (@total_ordering), а також використання селекторів attrgetter, itemgetter та methodcaller для побудови виразних функціональних конвеєрів обробки даних.

2. Теоретичні відомості

functools.partial: дозволяє зафіксувати частину позиційних або іменованих аргументів функції, повертаючи новий Callable-об’єкт зі спрощеною сигнатурою. Незамінний для конфігурації кроків конвеєра.

functools.singledispatch: механізм реалізації ad-hoc поліморфізму (перевантаження) на основі типу першого позиційного аргументу, що дозволяє створювати єдину точку входу для завантаження даних із рядків, шляхів файлів Path, потоків введення/виведення TextIO або готових колекцій.

functools.total_ordering: класовий декоратор, що автоматично генерує всі методи порівняння (__le__, __gt__, __ge__), якщо в класі реалізовано __eq__ та один з операторів відношення (наприклад __lt__). При порівнянні з несумісними типами методи повинні повертати NotImplemented.

Модуль operator: надає ефективні функції-геттери (attrgetter, itemgetter) та засіб виклику методів за назвою (methodcaller), що замінюють тривіальні lambda-вирази швидким C-кодом.

3. Постановка завдання (Варіант 6)

Тема варіанта 6: Замовлення інтернет-магазину.

Вхідні дані: Замовлення: id, customer, total, status, city.

Конвеєр / reduce: Відібрати оплачені замовлення (status == 'paid'), застосувати знижку через functools.partial, відсортувати за сумою спадання та id зростання, отримати Top-N; порахувати агреговану статистику через functools.reduce (кількість, виторг, середній чек, максимум).

Обов’язкові вимоги: load_orders через @singledispatch для підтримки str, Path, TextIO, list, з обробкою JSON та CSV; клас OrderRank з @total_ordering; використання operator.attrgetter та methodcaller.

4. Хід роботи та лістинг програми

Доменна модель та OrderRank (models.py)

```
from __future__ import annotations

from dataclasses import dataclass
from functools import total_ordering
from typing import Any


@dataclass(frozen=True, slots=True)
class Order:
    id: int
    customer: str
    total: float
    status: str
    city: str

    def discounted(self, pct: float) -> Order:
        new_total = round(max(0.0, self.total * (1.0 - pct)), 2)
        return Order(
            id=self.id,
            customer=self.customer,
            total=new_total,
            status=self.status,
            city=self.city,
        )


@total_ordering
class OrderRank:
    def __init__(self, order: Order) -> None:
        self.order = order

    @property
    def total(self) -> float:
        return self.order.total

    @property
    def order_id(self) -> int:
        return self.order.id

    def __eq__(self, other: Any) -> bool:
        if not isinstance(other, OrderRank):
            return NotImplemented
        return (self.total, self.order_id) == (other.total, other.order_id)

    def __lt__(self, other: Any) -> bool:
        if not isinstance(other, OrderRank):
            return NotImplemented
        if self.total != other.total:
            return self.total < other.total
        return self.order_id > other.order_id
```

```
    def __repr__(self) -> str:
        return f"OrderRank(id={self.order_id}, total={self.total})"
```

Поліморфний завантажувач (@singledispatch) (loaders.py)

```
from __future__ import annotations

import csv
import io
import json
from collections.abc import Iterable
from functools import singledispatch
from pathlib import Path
from typing import Any

from models import Order


def _dict_to_order(d: dict[str, Any]) -> Order:
    return Order(
        id=int(d["id"]),
        customer=str(d["customer"]),
        total=float(d["total"]),
        status=str(d["status"]),
        city=str(d["city"]),
    )


def parse_json_orders(content: str) -> list[Order]:
    data = json.loads(content)
    return [_dict_to_order(item) for item in data]


def parse_csv_orders(content: str) -> list[Order]:
    reader = csv.DictReader(io.StringIO(content.strip()))
    return [_dict_to_order(row) for row in reader]


@singledispatch
def load_orders(source: Any) -> list[Order]:
    raise TypeError(f"Unsupported source type: {type(source).__name__}")


@load_orders.register(str)
def _load_from_str(source: str) -> list[Order]:
    trimmed = source.strip()
    if trimmed.startswith("[") or trimmed.startswith("{"):
        return parse_json_orders(trimmed)
    p = Path(trimmed)
    if p.exists() and p.is_file():
        return load_orders(p)
    if "customer" in trimmed and "total" in trimmed:
        return parse_csv_orders(trimmed)
    raise ValueError(f"Cannot parse string source: {source[:30]}...")


@load_orders.register(Path)
def _load_from_path(source: Path) -> list[Order]:
    content = source.read_text(encoding="utf-8")
```

```
    if source.suffix.lower() == ".json":
        return parse_json_orders(content)
    if source.suffix.lower() == ".csv":
        return parse_csv_orders(content)
    try:
        return parse_json_orders(content)
    except Exception:
        return parse_csv_orders(content)


@load_orders.register(io.TextIOBase)
def _load_from_textio(source: io.TextIOBase) -> list[Order]:
    content = source.read()
    try:
        return parse_json_orders(content)
    except Exception:
        return parse_csv_orders(content)


@load_orders.register(list)
@load_orders.register(tuple)
def _load_from_iterable(source: Iterable[Any]) -> list[Order]:
    result: list[Order] = []
    for item in source:
        if isinstance(item, Order):
            result.append(item)
        elif isinstance(item, dict):
            result.append(_dict_to_order(item))
        else:
            raise TypeError(f"Expected Order or dict, got {type(item).__name__}")
    return result
```

Конвеєр та reduce-агрегація (core.py)

```
from __future__ import annotations

from collections.abc import Iterable
from functools import partial, reduce
from operator import attrgetter, methodcaller
from typing import TypedDict

from models import Order, OrderRank


class OrderStats(TypedDict):
    count: int
    revenue: float
    avg_total: float
    max_total: float


get_total = attrgetter("total")
get_status = attrgetter("status")


def is_paid(order: Order) -> bool:
    status_val = str(get_status(order))
    return status_val.lower() == "paid"
```

```
def filter_paid_orders(orders: Iterable[Order]) -> list[Order]:
    return [o for o in orders if is_paid(o)]


def apply_discount_rate(order: Order, rate: float) -> Order:
    caller = methodcaller("discounted", rate)
    return caller(order)  # type: ignore[no-any-return]


make_standard_discount = partial(apply_discount_rate, rate=0.05)


def sort_and_rank_orders(orders: Iterable[Order], top_n: int = 5) -> list[Order]:
    ranked = sorted(
        [OrderRank(o) for o in orders],
        reverse=True,
    )
    return [r.order for r in ranked[:top_n]]


def calculate_orders_stats(orders: Iterable[Order]) -> OrderStats:
    order_list = list(orders)
    if not order_list:
        return {
            "count": 0,
            "revenue": 0.0,
            "avg_total": 0.0,
            "max_total": 0.0,
        }

    totals = [float(get_total(o)) for o in order_list]

    initial: tuple[int, float, float] = (0, 0.0, float("-inf"))

    def acc_fn(acc: tuple[int, float, float], val: float) -> tuple[int, float, float]:
        cnt, rev, max_t = acc
        return cnt + 1, round(rev + val, 2), max(max_t, val)

    count, revenue, max_val = reduce(acc_fn, totals, initial)
    avg_val = round(revenue / count, 2) if count > 0 else 0.0

    return {
        "count": count,
        "revenue": revenue,
        "avg_total": avg_val,
        "max_total": max_val,
    }


def process_orders_pipeline(
    orders: Iterable[Order],
    discount_rate: float = 0.05,
    top_n: int = 5,
) -> tuple[list[Order], OrderStats]:
    paid = filter_paid_orders(orders)
    discount_fn = partial(apply_discount_rate, rate=discount_rate)
    discounted = [discount_fn(o) for o in paid]
    top_orders = sort_and_rank_orders(discounted, top_n=top_n)
```

```
    stats = calculate_orders_stats(top_orders)
    return top_orders, stats
```

CLI точка входу (main.py)

```
from __future__ import annotations

import argparse
from pathlib import Path

from core import process_orders_pipeline
from loaders import load_orders
from models import Order


def print_orders_table(orders: list[Order]) -> None:
    print(f"{'ID':<4} {'Клієнт':<20} {'Місто':<12} {'Сума (грн)':<12}")
    print("-" * 52)
    for o in orders:
        print(f"{o.id:<4} {o.customer:<20} {o.city:<12} {o.total:<12.2f}")


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--file", type=Path, default=Path("sample_data.json"))
    parser.add_argument("--top", type=int, default=5)
    parser.add_argument("--discount", type=float, default=0.05)

    args = parser.parse_args()

    orders = load_orders(args.file)
    top_orders, stats = process_orders_pipeline(
        orders,
        discount_rate=args.discount,
        top_n=args.top,
    )

    print("=" * 52)
    print("      РЕЙТИНГ ОПЛАЧЕНИХ ЗАМОВЛЕНЬ (ВАРІАНТ 6)")
    print("=" * 52)
    print_orders_table(top_orders)
    print("=" * 52)
    print("СТАТИСТИКА (REDUCE):")
    print(f"  Кількість:     {stats['count']}")
    print(f"  Сумарний виторг: {stats['revenue']:.2f} грн")
    print(f"  Середній чек:   {stats['avg_total']:.2f} грн")
    print(f"  Максимальний чек: {stats['max_total']:.2f} грн")
    print("=" * 52)


if __name__ == "__main__":
    main()
```

Тести компонентів та конвеєра (tests/test_lab07.py)

```
import io
from pathlib import Path
```

```
import pytest
from core import (
    calculate_orders_stats,
    make_standard_discount,
    process_orders_pipeline,
)
from loaders import load_orders
from models import Order, OrderRank


def test_order_rank_total_ordering() -> None:
    o1 = Order(1, "A", 1000.0, "paid", "Київ")
    o2 = Order(2, "B", 2000.0, "paid", "Львів")
    o3 = Order(3, "C", 1000.0, "paid", "Одеса")

    r1, r2, r3 = OrderRank(o1), OrderRank(o2), OrderRank(o3)

    assert r1 < r2
    assert r2 > r1
    assert r1 <= r2
    assert r2 >= r1
    assert r1 == OrderRank(Order(1, "A", 1000.0, "paid", "Київ"))

    assert r1 > r3
    assert r3 < r1

    assert (r1 == "not_a_rank") is False
    with pytest.raises(TypeError):
        _ = r1 < 123


def test_singledispatch_str_json() -> None:
    raw_json = (
        '[{"id": 10, "customer": "Test", '
        '"total": 100.0, "status": "paid", "city": "Київ"}]'
    )
    orders = load_orders(raw_json)
    assert len(orders) == 1
    assert orders[0].id == 10
    assert orders[0].total == 100.0


def test_singledispatch_str_csv() -> None:
    raw_csv = "id,customer,total,status,city\n11,Anna,300.0,paid,Львів\n"
    orders = load_orders(raw_csv)
    assert len(orders) == 1
    assert orders[0].id == 11
    assert orders[0].customer == "Anna"


def test_singledispatch_files(tmp_path: Path) -> None:
    json_file = tmp_path / "orders.json"
    json_file.write_text(
        '[{"id": 20, "customer": "Petro", "total": 500.0, '
        '"status": "paid", "city": "Львів"}]',
        encoding="utf-8",
    )
    from_path = load_orders(json_file)
    assert len(from_path) == 1
    assert from_path[0].id == 20
```

```
    csv_file = tmp_path / "orders.csv"
    csv_file.write_text(
        "id,customer,total,status,city\n21,Oleg,750.0,paid,Київ\n",
        encoding="utf-8",
    )
    from_csv_path = load_orders(csv_file)
    assert len(from_csv_path) == 1
    assert from_csv_path[0].id == 21


def test_singledispatch_textio() -> None:
    buf = io.StringIO("id,customer,total,status,city\n30,Dmytro,900.0,paid,Харків\n")
    orders = load_orders(buf)
    assert len(orders) == 1
    assert orders[0].id == 30


def test_singledispatch_list_collection(
    sample_order_list: list[Order],
) -> None:
    orders = load_orders(sample_order_list)
    assert len(orders) == len(sample_order_list)


def test_singledispatch_unsupported() -> None:
    with pytest.raises(TypeError):
        load_orders(12345)


def test_pipeline_processing(sample_order_list: list[Order]) -> None:
    top, stats = process_orders_pipeline(sample_order_list, discount_rate=0.1, top_n=2)
    assert len(top) == 2
    assert stats["count"] == 2
    assert top[0].total == 2250.0
    assert top[1].total == 900.0


def test_partial_discount() -> None:
    order = Order(1, "Test", 100.0, "paid", "Київ")
    discounted = make_standard_discount(order)
    assert discounted.total == 95.0


def test_empty_orders_stats() -> None:
    stats = calculate_orders_stats([])
    assert stats["count"] == 0
    assert stats["revenue"] == 0.0
```

Property-based тести (Hypothesis) (tests/test_properties.py)

```
import copy
from collections.abc import Sequence

from core import process_orders_pipeline
from hypothesis import given
from hypothesis import strategies as st
from models import Order
```

```
order_strategy = st.builds(
    Order,
    id=st.integers(min_value=1, max_value=10000),
    customer=st.text(min_size=1, max_size=20),
    total=st.floats(
        min_value=10.0, max_value=10000.0, allow_nan=False, allow_infinity=False
    ),
    status=st.sampled_from(["paid", "pending", "cancelled"]),
    city=st.sampled_from(["Львів", "Київ", "Одеса", "Харків"]),
)


@given(st.lists(order_strategy, min_size=0, max_size=20))
def test_property_no_mutation(orders: Sequence[Order]) -> None:
    snapshot = copy.deepcopy(list(orders))
    _ = process_orders_pipeline(orders)
    assert orders == snapshot


@given(st.lists(order_strategy, min_size=0, max_size=20))
def test_property_invariants(orders: Sequence[Order]) -> None:
    top_orders, stats = process_orders_pipeline(orders, top_n=5)
    assert len(top_orders) <= 5
    assert stats["count"] == len(top_orders)
    if stats["count"] > 0:
        assert stats["revenue"] >= 0.0
        assert stats["avg_total"] <= stats["max_total"]
        for o in top_orders:
            assert o.status.lower() == "paid"
```

5. Результати виконання та тестування

Запуск демонстраційної програми main.py

```
$ python main.py --file sample_data.json --top 5 --discount 0.05
==================================================
      РЕЙТИНГ ОПЛАЧЕНИХ ЗАМОВЛЕНЬ (ВАРІАНТ 6)
==================================================
ID   Клієнт               Місто        Сума (грн)
--------------------------------------------------
6    Наталія Мороз        Київ         2945.00
3    Олег Бондар          Львів        2660.00
1    Іван Коваль          Львів        1187.50
4    Оксана Ткач          Одеса        1187.50
7    Сергій Кравчук       Харків       570.00
==================================================
СТАТИСТИКА (REDUCE):
  Кількість:     5
  Сумарний виторг: 8550.00 грн
  Середній чек:   1710.00 грн
  Максимальний чек: 2945.00 грн
==================================================
```

Результати виконання тестів pytest

```
$ pytest -q
............                                                             [100%]
12 passed in 1.61s
```

Статична перевірка типів mypy --strict

```
$ mypy --strict .
Success: no issues found in 7 source files
```

Перевірка форматування ruff та black

```
$ ruff check .
All checks passed!

$ black --check .
All done! ✨ 🍰 ✨
7 files would be left unchanged.
```

6. Висновки

У лабораторній роботі № 7 реалізовано надійний конвеєр обробки замовлень інтернет-магазину. Застосування @singledispatch дозволило створити універсальний

шар зчитування джерел даних (файлів JSON, CSV, потоків та пам’яті). Завдяки @total_ordering та класу OrderRank реалізовано багатокритеріальне ранжування, а модуль operator забезпечив високу лаконічність і швидкість роботи конвеєра.